

CORDOVA

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Uwaga	1
Instalacja Cordova	2
Utworzenie projektu Cordova i pierwsze uruchomienie.....	2
Podmiana projektu domyślnego na Pogodynę.....	4
Android.....	7
Electron i aplikacja na Windows	11
Debuggowanie aplikacji Electron	13
Dostosowanie aplikacji Electron	14
Budowa aplikacji	17
Commit projektu do GIT.....	20
Podsumowanie.....	20

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- tworzenie hybrydowych aplikacji mobilnych z wykorzystaniem oprogramowania Apache Cordova;
- reużywanie istniejącego kodu HTML do tworzenia aplikacji mobilnych.

W praktycznym wymiarze uczestnicy zamienią swoją aplikację pogodową z LAB D na prostą aplikację mobilną Android.

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do **Plik** -> **Informacje** -> **Właściwości** -> **Właściwości zaawansowane** -> **Niestandardowe** i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub **Ctrl+A** -> **F9**.

INSTALACJA CORDOVA

Upewnij się, że w systemie operacyjnym masz poprawnie zainstalowany lub rozpakowany i skonfigurowany menedżer pakietów NPM. Był on już wielokrotnie wykorzystywany, więc powinien być gotowy (por. LAB E, LAB F).

Otwórz ulubiony terminal i zainstaluj oprogramowanie Apache Cordova globalnie dla Twojej instancji NPM, poprzez wykonanie polecenia:

```
> npm install -g cordova
...
added 547 packages in 32s
```

UTWORZENIE PROJEKTU CORDOVA I PIERWSZE URUCHOMIENIE

Utwórz i wejdź poprzez terminal do katalogu `C:\Users\...\Desktop\ai1-cordova`. Następnie utwórz swój projekt z wykorzystaniem polecenia:

```
> cordova create c55621
Creating a new cordova project.
```

Otwórz utworzony katalog `c55621` w Visual Studio Code. Zapoznaj się ze strukturą katalogów.

Omów w maksymalnie 100 słowach zawartość projektu na tym etapie:

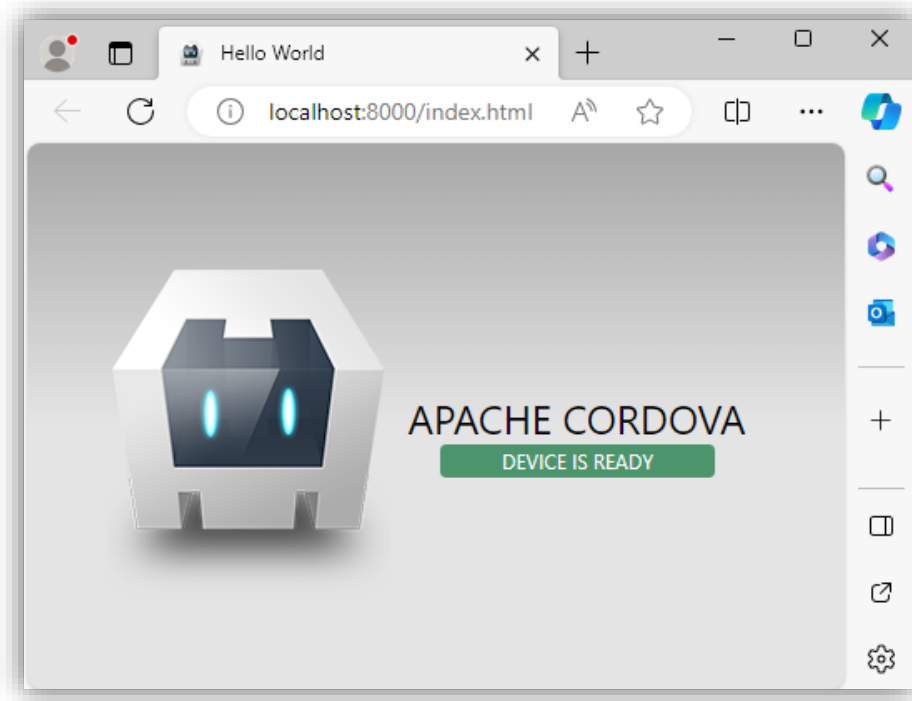
Projekt składa się z głównego katalogu `www`, w którym znajduje się kod źródłowy naszej aplikacji: plik startowy `index.html` oraz foldery na style, obrazy i skrypty. To tutaj mieści się cała warstwa wizualna.

Poza tym w folderze głównym widać plik `config.xml`, który jest kluczowy do konfiguracji ustawień Cordovy, oraz plik `package.json`, który służy do zarządzania bibliotekami i zależnościami projektu. Struktura przypomina standardową stronę internetową przygotowaną do konwersji na aplikację mobilną.

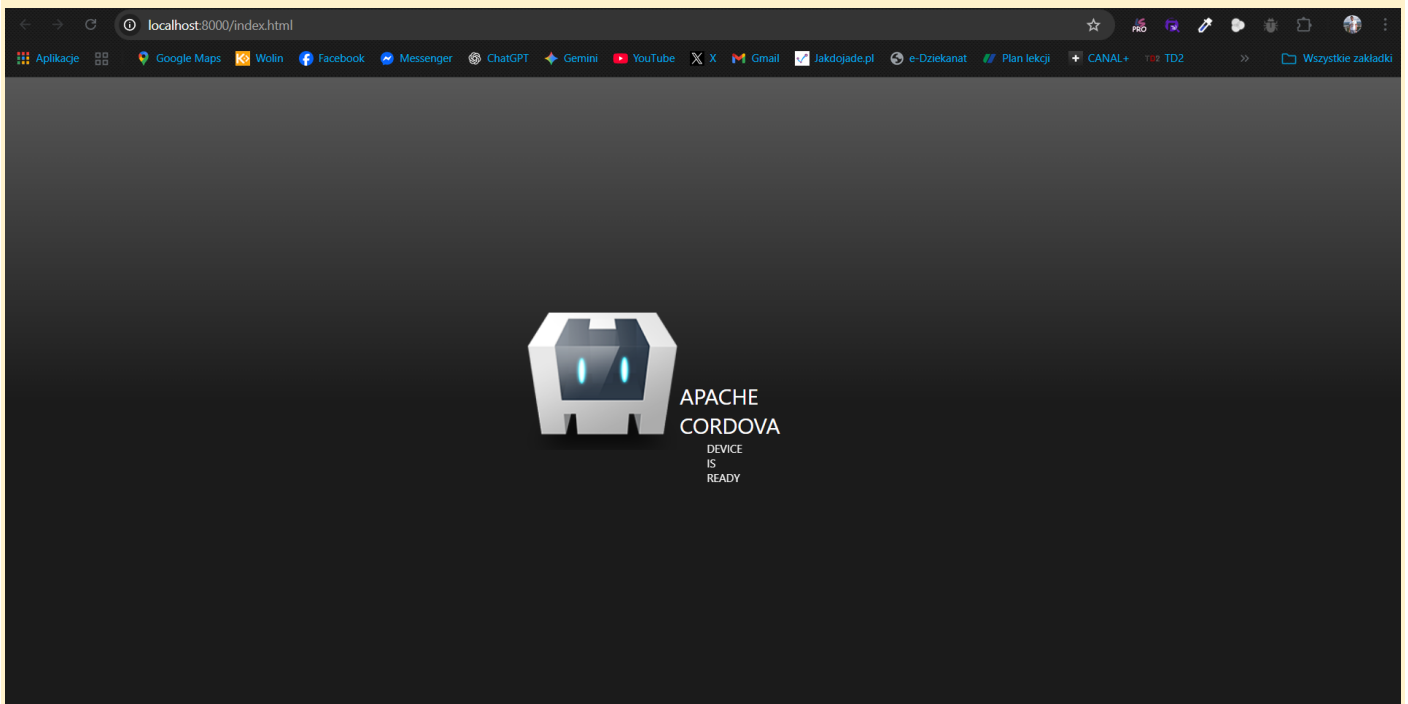
Następnie wróć do terminala, wejdź do katalogu projektu i dodaj platformę `browser`, po czym uruchom emulację w przeglądarce:

```
> cd c55621
> cordova platform add browser
> cordova emulate browser
```

Powinna uruchomić się przeglądarka z domyślnym projektem:



Wstaw zrzut ekranu emulacji w przeglądarce domyślnego projektu:



Wstaw zrzut ekranu zawartości terminalu po uruchomieniu emulacji i wyświetleniu strony:

```

Path: C:\xampp\htdocs\aplikacje-internetowe1\lab-v\c55621\platforms\browser
Name: Hello Cordova
PS C:\xampp\htdocs\aplikacje-internetowe1\lab-v\c55621> cordova emulate browser
startPage = index.html
Static file server running @ http://localhost:8000/index.html
CTRL + C to shut down
200 /index.html (br)
200 /css/index.css (br)
200 /cordova.js (br)
200 /img/logo.png
404 /cordova_plugins.js
200 /js/index.js (br)
200 /favicon.ico (br)

```

Na jakim porcie uruchomiła się Tobie emulacja?

8000

W maksymalnie 100 słowach opisz zmiany, które zaszły w strukturze katalogów projektu:

Po dodaniu platformy przeglądarkowej struktura projektu wyraźnie się rozbudowała. Najważniejszą zmianą jest pojawienie się katalogu platforms, w którym utworzył się folder browser, to tam Cordova przechowuje pliki specyficzne dla uruchamiania aplikacji w przeglądarce.

Dodatkowo w głównym katalogu pojawił się folder node_modules, zawierający wszystkie biblioteki i zależności niezbędne do działania projektu. Doszedł również plik package-lock.json, który precyzyjnie zapisuje wersje zainstalowanych pakietów, co zapewnia spójność środowiska.

Punkty:	0	1
---------	---	---

PODMIANA PROJEKTU DOMYŚLNEGO NA POGODYNKĘ

Zatrzymaj emulację w przeglądarce, jeśli jest jeszcze uruchomiona, z wykorzystaniem skrótu klawiszowego **Ctrl + C**.

Umieść w katalogu www projektu pliki HTML, JS i CSS z klientem REST do wyświetlania pogody, które opracowane zostały w ramach laboratorium LAB D. Upewnij się, że **główny plik HTML ma nazwę weather.html**. Upewnij się, że w pliku HTML w dalszym ciągu są poprawne ścieżki względne do plików JS i CSS. Upewnij się, że w kodzie użyty jest **Twój indywidualny klucz** do API.

Edytuj plik config.xml. Zmień domyślny plik projektu w znaczniku `<content>` z index.html na weather.html:

```
<content src="weather.html" />
```

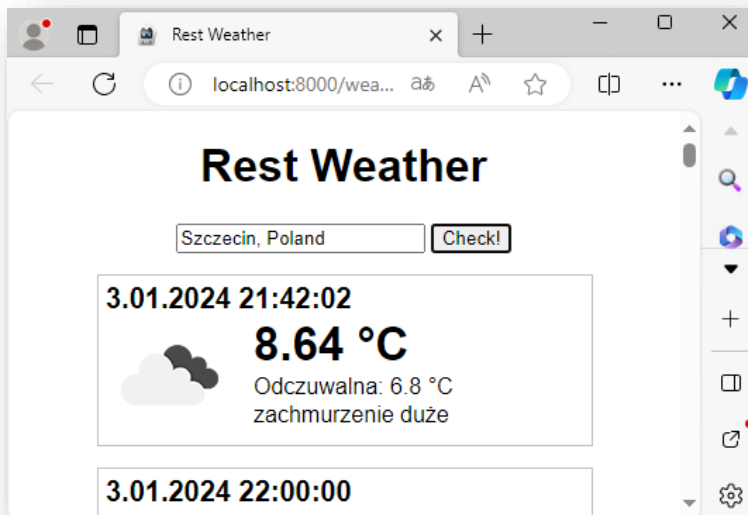
Zmień także identyfikator, nazwę, opis i autora aplikacji w pliku `config.xml`. Nazwa aplikacji musi składać się z Twojego imienia i numeru indeksu. Identyfikatorem musi być odwrócona nazwa domenowa WI wraz z Twoim identyfikatorem LDAP. Przykładowo:

```
<widget id="pl.edu.zut.wi.ni55621" ..... >
<name>Jakub 55621</name>
<description>Weather App</description>
<author email="ni00000@zut.edu.pl" href="https://www.wi.zut.edu.pl">
  Artur Karczmarczyk
</author>
```

Po wprowadzeniu zmian, zapisz wszystkie pliki i ponownie uruchom emulację projektu w przeglądarce:

```
> cordova emulate browser
```

Po uruchomieniu emulacji, w przeglądarce powinna pojawić się aplikacja Pogodynki. Wprowadź nazwę miejscowości i wyszukaj prognozy pogody, przykładowo:



Wstaw zrzut ekranu emulacji w przeglądarce projektu Pogodynki przed wyszukaniem prognoz:

Sprawdź pogodę

Wpisz nazwę miasta (np. Wrocław)

Pogoda

Wstaw zrzut ekranu emulacji pogodynki po wyszukaniu prognoz:



Wstaw zrzut ekranu pliku config.xml:

```
config.xml X
c55621 > config.xml
1  <?xml version='1.0' encoding='utf-8'?>
2  <widget id="pl.edu.zut.wi.wj55621" version="1.0.0" xmlns="http://www.w3.org/ns/widgets" xmlns:cdv="http://cordova.apache.
3      <name>Jakub 55621</name>
4      <description>
5          Weather App
6      </description>
7      <author email="wj55621@zut.edu.pl" href="https://www.wi.zut.edu.pl">
8          Jakub Wierciński
9      </author>
10     <content src="weather.html" />
11     <allow-intent href="http://*/*" />
12     <allow-intent href="https://*/*" />
13 </widget>
14
```

Wstaw zrzut ekranu fragmentu kodu zawierającego wykorzystanie Twojego klucza do API. Upewnij się, że klucz jest widoczny i że jest to Twój własny klucz:

```

c55621 > www > JS script.js > ...
1  const apiKey = '4ca8aa15d7312c84bc1afd1b85bd79b9';
2
3  document.getElementById('getWeatherBtn').addEventListener('click', function() {
4      const city = document.getElementById('cityInput').value;
5
6      if(!city) {
7          alert("Wpisz nazwę miasta!");
8          return;
9      }
10
11     document.getElementById('currentWeatherSection').classList.remove('hidden');
12     document.getElementById('forecastSection').classList.remove('hidden');
13
14     document.getElementById('current-weather').innerHTML = 'Pobieranie danych...';
15     document.getElementById('forecast-container').innerHTML = 'Pobieranie danych...';
16
17     getCurrentWeatherXHR(city);
18     getForecastFetch(city);
19 });
20
21 //XMLHttpRequest

```

Punkty:	0	1
---------	---	---

ANDROID

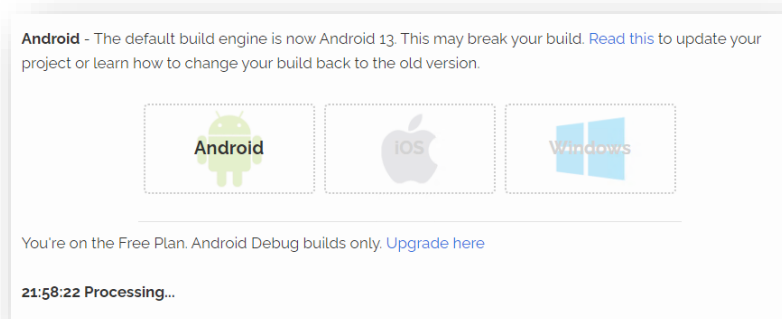
W tej sekcji wykorzystamy projekt Cordova do zbudowania aplikacji Pogodynka na Android. Do budowy wykorzystamy platformę VoltBuilder.

Najpierw w konsoli dodaj platformę Android:

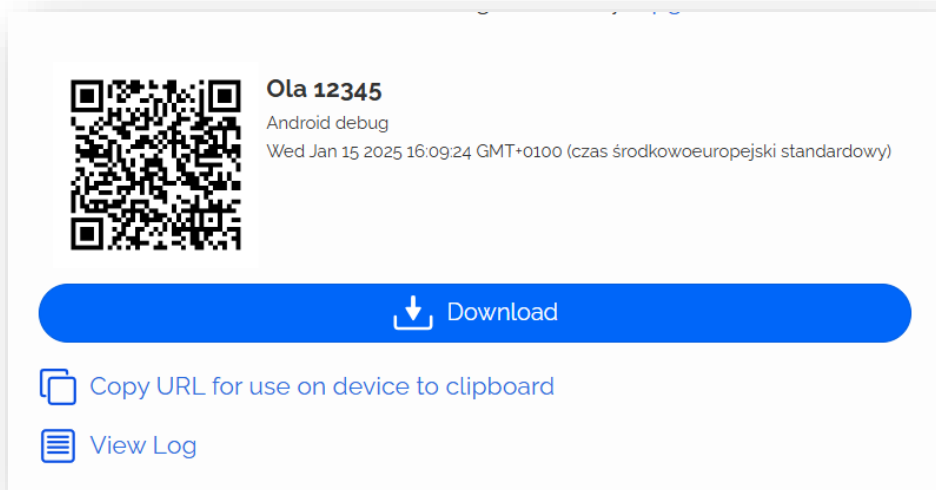
```
> cordova platform add android
```

Następnie utwórz archiwum ZIP z całym katalogiem projektu, tj. `C:\Users\...\Desktop\ai1-cordova\c55621`. Załóżmy, że otrzymane archiwum to `c55621.zip`.

Wejdź na stronę <https://volt.build>. Zarejestruj i zaloguj się. Utworzone zostanie darmowe konto. Przejdź do sekcji Upload, tj. <https://volt.build/upload/>. Wybierz Android i wgraj swoje archiwum ZIP. Poczekaj, aż aplikacja zostanie zbudowana:

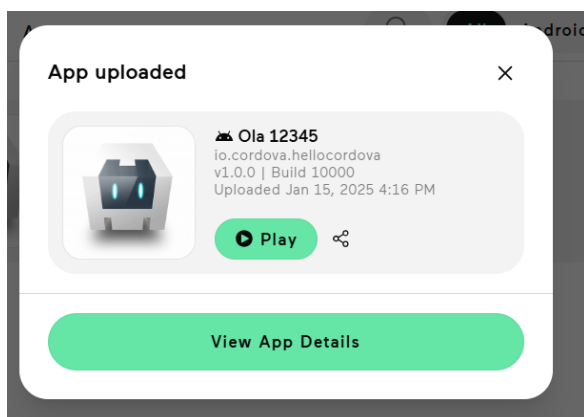


Po kilku minutach aplikacja będzie gotowa do pobrania:

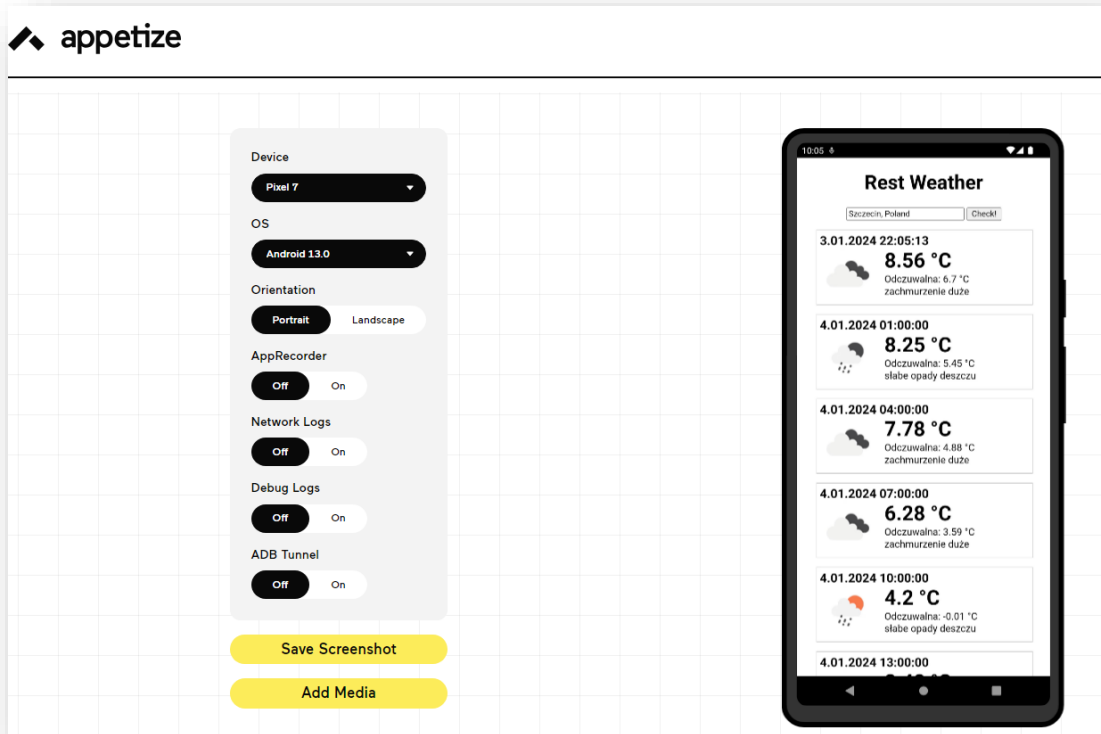


Pobierz aplikację. Będzie to plik .apk.

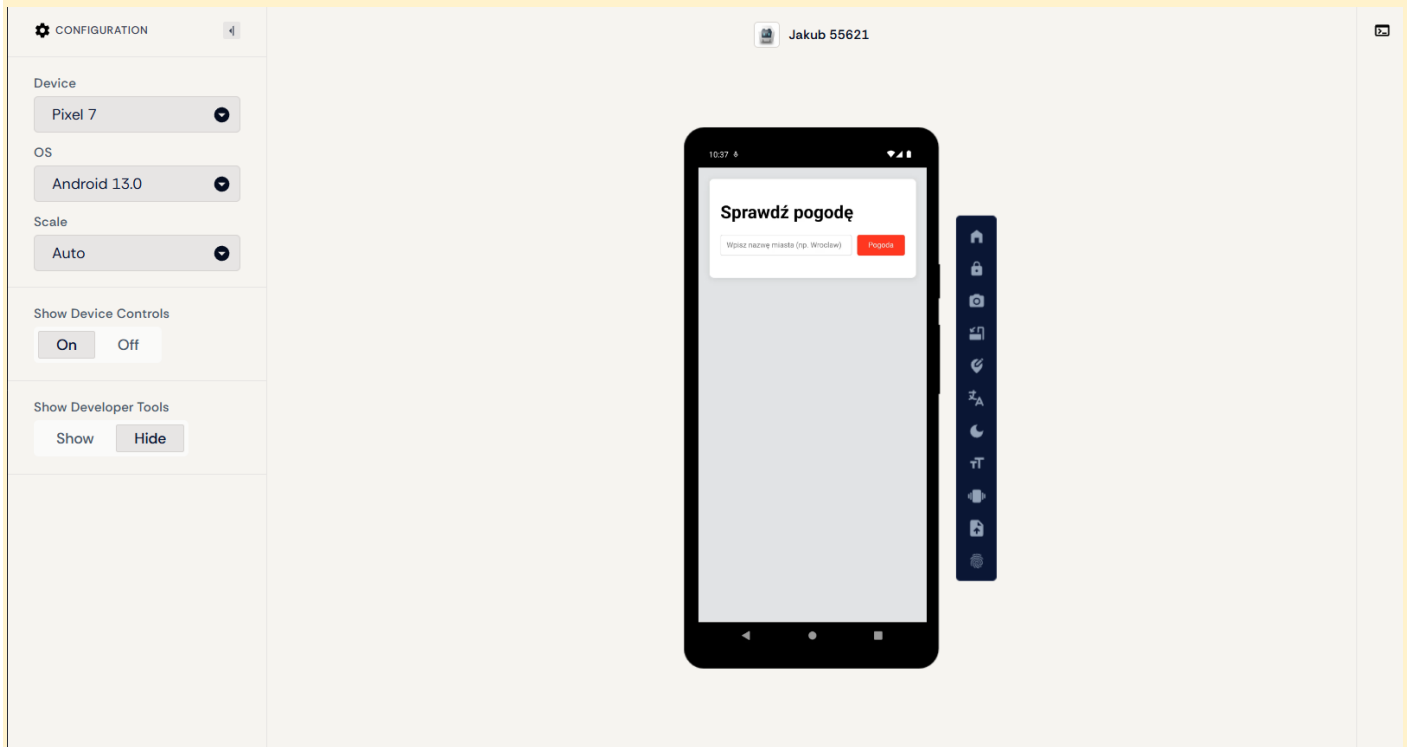
Teraz jesteśmy gotowi do przetestowania działania aplikacji w emulatorze Android. Wykorzystamy do tego celu usługę appetize. Wejdź na stronę <https://appetize.io/>. Utwórz darmowe konto. Przejdź do **Apps**. Przyciśnij **Upload App** i wybierz z dysku swoją aplikację. W ciągu kilku chwil otworzy się okno z informacją **App uploaded**:



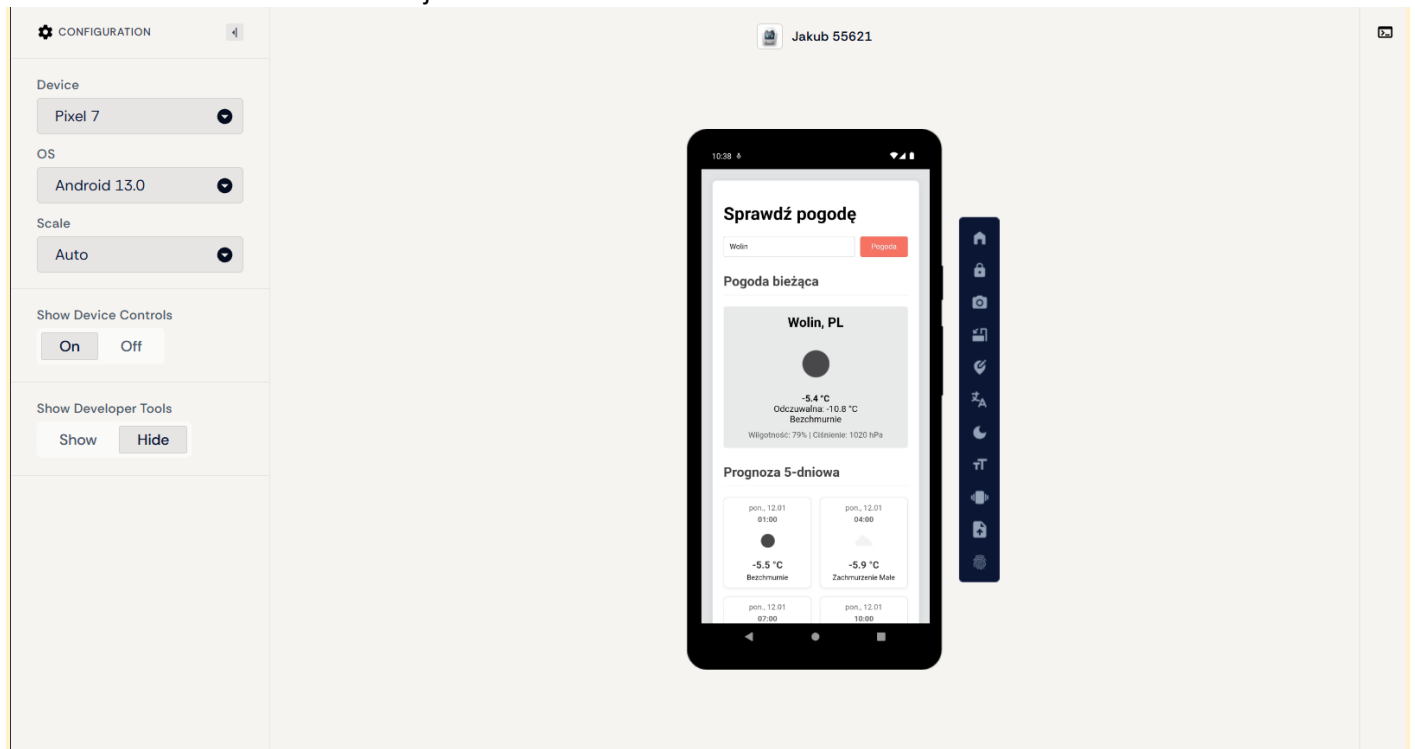
Kliknij przycisk Play. Wybierz urządzenie, system operacyjny i uruchom emulację:



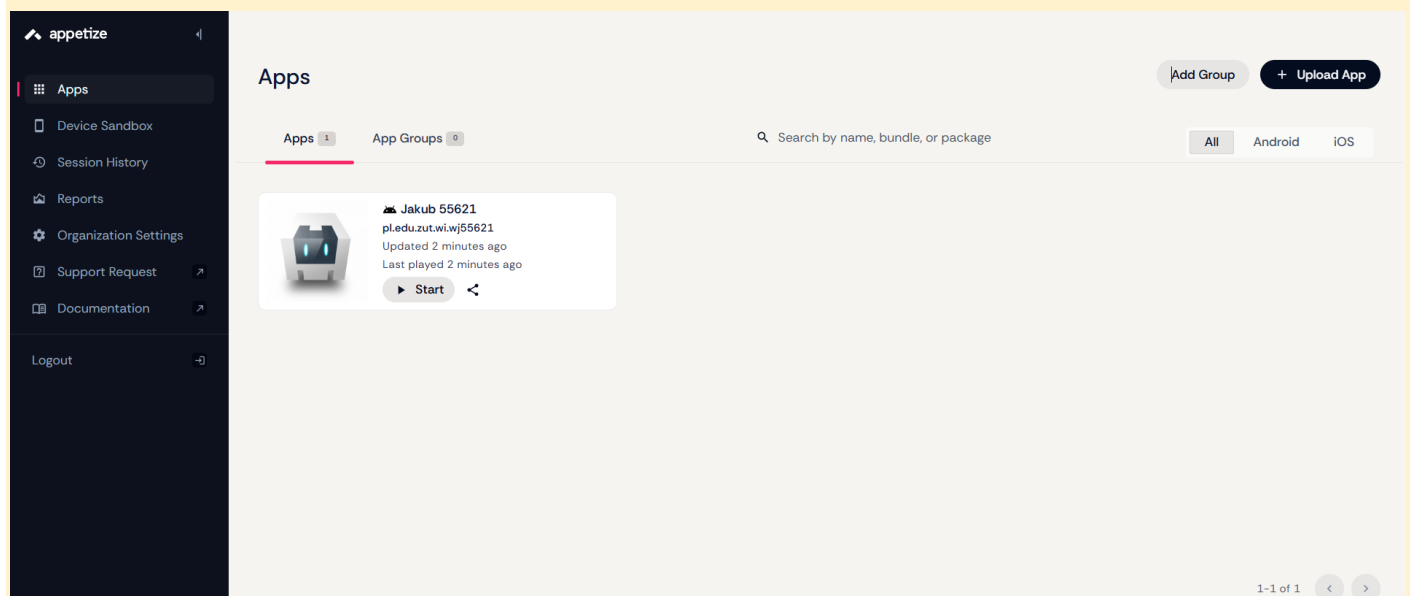
Wstaw zrzut ekranu Twojej aplikacji w **appetize** przed wyszukaniem prognozy:



Wstaw zrzut ekranu Twojej aplikacji w **appetize** po wyszukaniu prognozy:



Wstaw zrzut ekranu z załadowaną aplikacją **appetize** (okno z informacją App Uploaded / listę aplikacji). Upewnij się, że widoczna jest Twoja aplikacja a przy niej jest Twoje imię i numer albumu:



Wstaw zrzut ekranu fragmentu strony **VoltBuilder** przedstawiający zbudowaną aplikację. Upewnij się, że widoczne jest Twoje imię i numer albumu:

button to select

Android iOS Windows

You're on the Free Plan. Android Debug builds only. [Upgrade here](#)

Jakub 55621
Android debug
Sun Jan 11 2026 22:27:15 GMT+0100 (czas środkowoeuropejski standardowy)

[Download](#)

[Copy URL for use on device to clipboard](#)

[View Log](#)

Build ID: `fb9d7b38-00ba-4e91-a9b5-1bdfc5a576a` [Copy Build Id](#)

Punkty:

0

1

Rada!

Jeśli masz urządzenie Android, możesz zeskanować telefonem kod QR wyświetlony w **VoltBuilder**. Pod podanym adresem znajdziesz aplikację **.apk** do pobrania na Twój telefon. Następnie możesz zainstalować tę aplikację w telefonie i korzystać na co dzień!

ELECTRON I APLIKACJA NA WINDOWS

Dodaj do projektu platformę electron:

```
> cordova platform add electron
```

Przejdź do Visual Studio Code. W głównym katalogu projektu utwórz plik `build.json` o następującej zawartości:

```
{
  "electron": {
    "windows": {
      "package": [
        "zip"
      ]
    }
  }
}
```

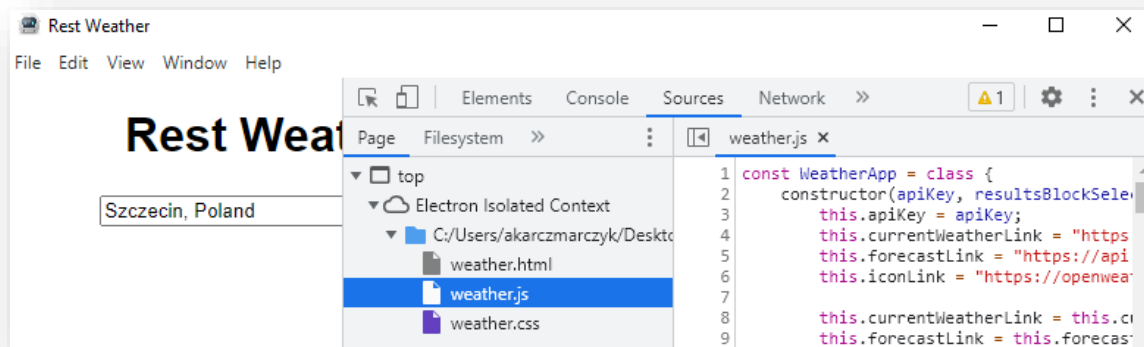
Przy budowie aplikacji, biblioteka Electron tworzyć będzie archiwum ZIP z projektem, zamiast instalatora. Dostępne postaci pakietów instalacyjnych opisane są tutaj:

<https://cordova.apache.org/docs/en/12.x/guide/platforms/electron/index.html#adding-a-package>.

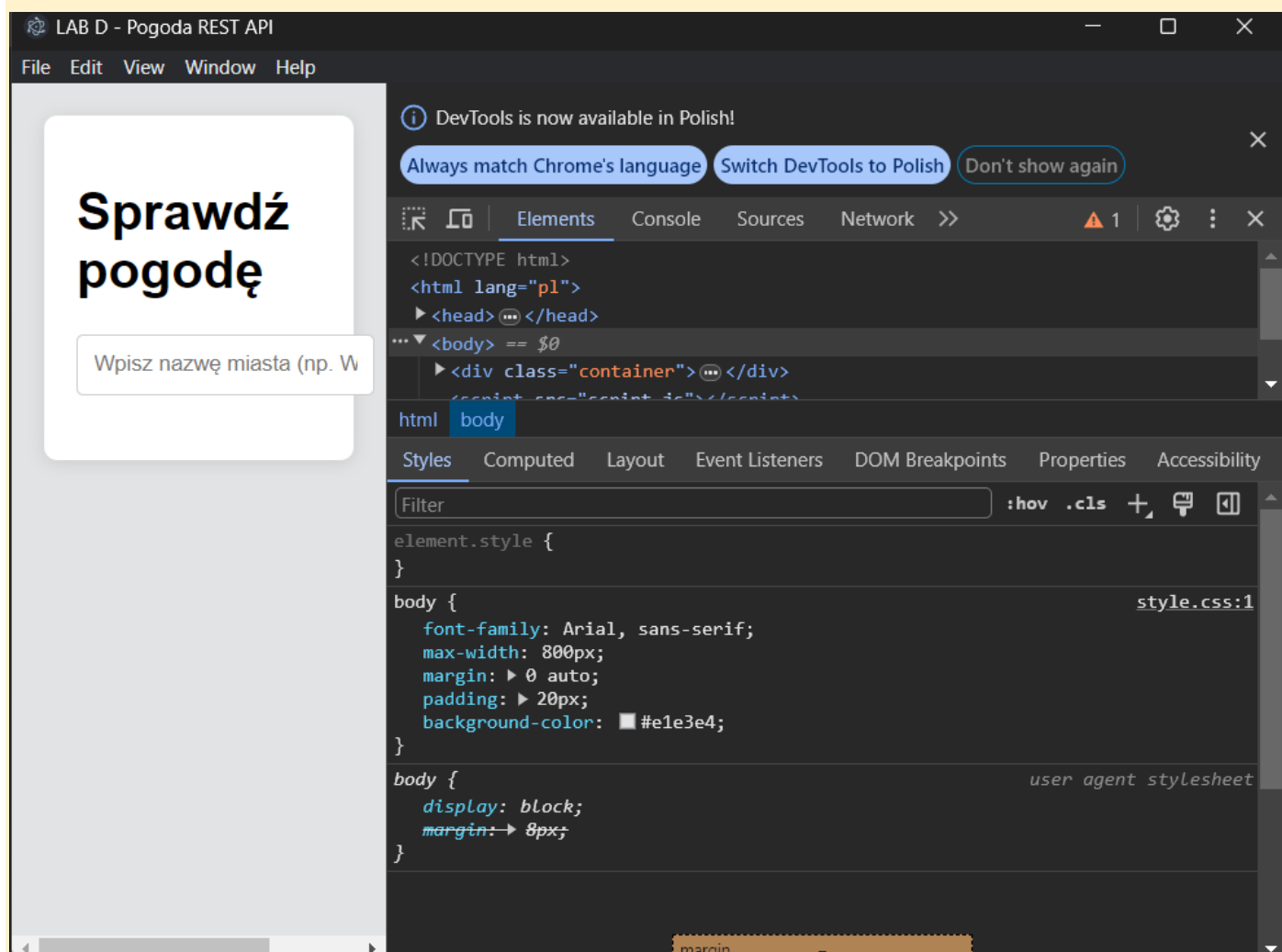
Uruchom teraz emulację aplikacji Electron:

```
> cordova emulate electron --nobuild
```

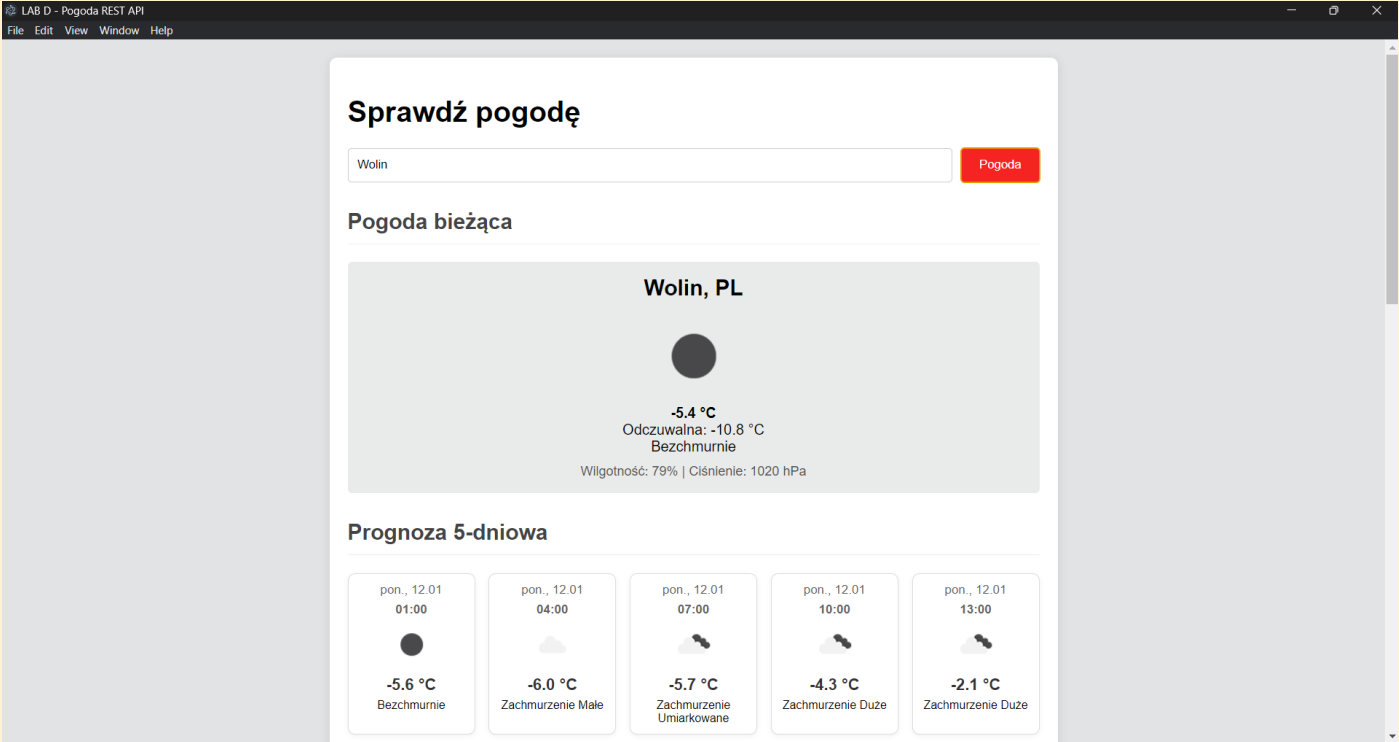
W efekcie uruchomi się aplikacja Electron:



Wstaw zrzut ekranu uruchomionej aplikacji Electron. Upewnij się że widoczne będzie całe okno wraz z ikoną Apache Cordova



Zamknij narzędzia deweloperskie (panel z prawej strony) i wyszukaj prognozę pogody. Upewnij się że widoczne będzie całe okno wraz z ikoną Apache Cordova. Wstaw zrzut ekranu:

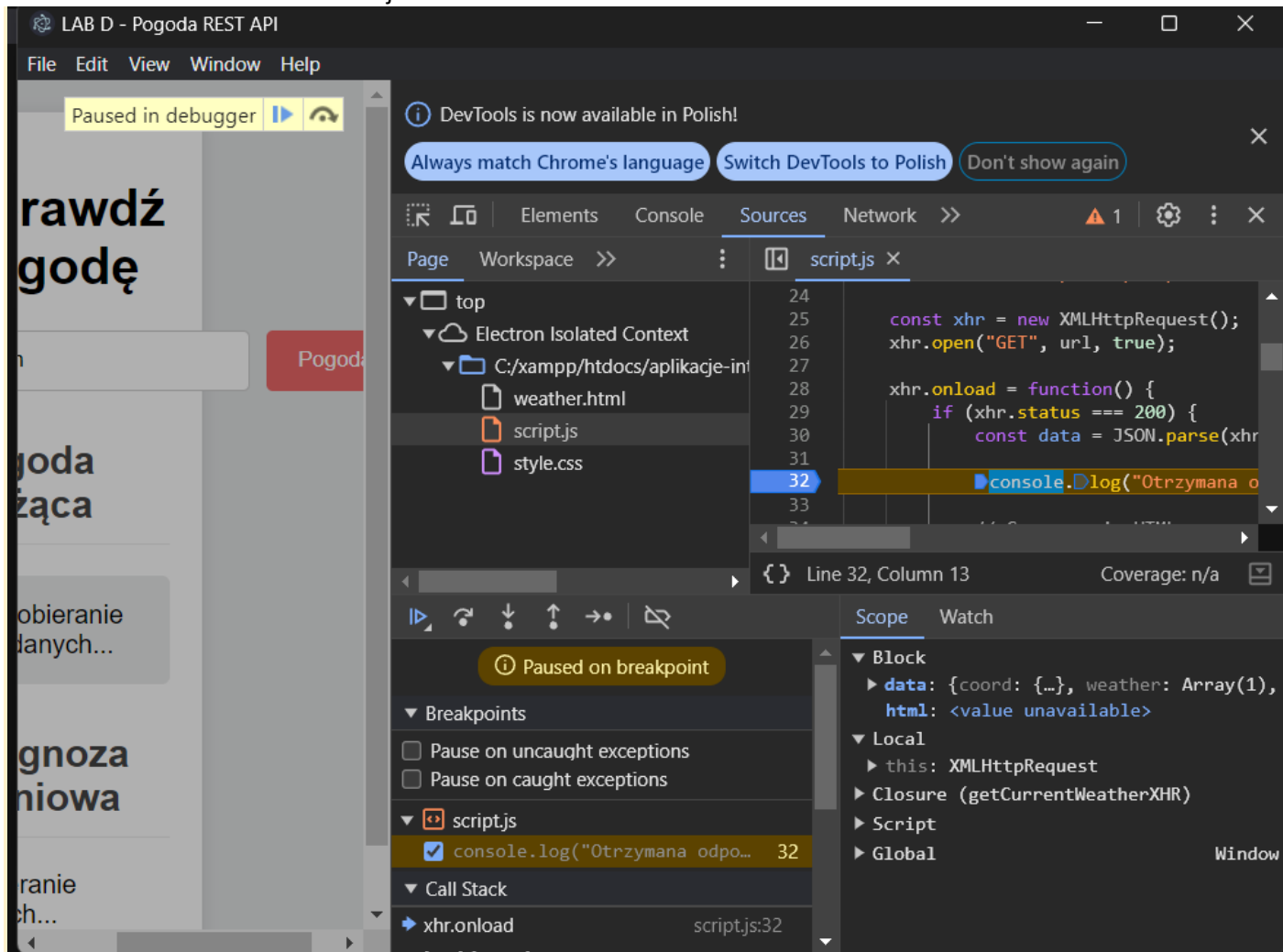


Punkty:	0	1
---------	---	---

DEBUGGOWANIE APLIKACJI ELECTRON

Ponownie włącz emulację aplikacji electron. W narzędziach deweloperskich (panel z prawej strony), wybierz zakładkę „Sources”. Znajdź kod JS odpowiedzialny za pobranie bieżącej prognozy pogody i ustaw pułapkę (ang. breakpoint) na w linii odpowiedniej funkcji. Wyszukaj prognozę pogody. Upewnij się, że pułapka się aktywowała.

Wstaw zrzut ekranu całości okienka electron z aplikacją zatrzymaną na breakpoincie:



Upewnij się, że widoczne są:

- nad treścią aplikacji informacja, że wykonanie zostało wstrzymane w debuggerze
- debugger zatrzymany na pułapce (linia podświetlona na niebiesko / żółto)
- w okienku Scope widoczne są wartości zmiennych

Punkty:	0	1
---------	---	---

DOSTOSOWANIE APLIKACJI ELECTRON

W Visual Studio Code utwórz w projekcie podkatalog `res\electron`. Utwórz w dowolnym programie graficznym ikonkę aplikacji o wymiarach **512x512 pikseli (format PNG)**. Jeśli do stworzenia ikony korzystasz z Chat GPT, dodaj do niej swoje inicjały lub imię. Utworzony plik zapisz w katalogu `res\electron`, a następnie w pliku `config.xml` skonfiguruj ikonę dla platformy electron, przykładowo:

```
<platform name="electron">
  <icon src="res/electron/cloud-sun.png" />
</platform>
```

Następnie utwórz plik `res\electron\settings.json` i zarejestruj go w `config.xml`:

```
<platform name="electron">
  <icon src="res/electron/cloud-sun.png" />
  <preference name="ElectronSettingsFilePath" value="res/electron/settings.json" />
</platform>
```

</platform>

W pliku `settings.json` dostosuj szerokość i wysokość aplikacji, a także ukryj menu. Dokumentacja: <https://cordova.apache.org/docs/en/12.x/guide/platforms/electron/index.html#customizing-the-application's-window-options>

Przykładowa zawartość pliku `settings.json`:

```
{
  "browserWindow": {
    "width": 1024,
    "height": 600,
    "autoHideMenuBar": true
  }
}
```

Ponownie dokonaj emulacji aplikacji, tym razem z przełącznikiem `--release`, który ukryje narzędzia deweloperskie.

Uwaga! W przypadku niektórych systemów może wystąpić błąd:

Cannot create symbolic link : A required privilege is not held by the client.

Szczegółowy opis rozwiązania znajduje się tu:

<https://stackoverflow.com/questions/78560953/i-am-get-this-error-while-build-my-electron-vite-project>

W skrócie:

- pobierz plik <https://github.com/electron-userland/electron-builder-binaries/releases/download/winCodeSign-2.6.0/winCodeSign-2.6.0.7z>
- rozpakuj go do katalogu zgodnego z komunikatem błędu, np. `C:\Users\artur\AppData\Local\electron-builder\Cache\winCodeSign\winCodeSign-2.6.0`
- uruchom ponownie budowanie aplikacji

Jak nic nie działa (absolutnie nie polecane rozwiązanie):

- uruchom terminal jako administrator i spróbuj ponownie (nie da się w sieci ZUT)

Wstaw zrzut ekranu kodu pliku `settings.json`:

```
config.xml  settings.json X  JS script.js  {} build.json

c55621 > res > electron > {} settings.json > ...
1  {
2    "browserWindow": {
3      "width": 1024,
4      "height": 600,
5      "autoHideMenuBar": true
6    }
7  }
```

Wstaw zrzut ekranu kodu pliku `config.xml`:

```
config.xml X {} settings.json JS script.js {} build.json
c55621 > config.xml
1  <?xml version='1.0' encoding='utf-8'?>
2  <widget id="pl.edu.zut.wi.wj55621" version="1.0.0" xmlns="http://www.w3.org/ns/widgets" xmlns:cdv="http://cordova.apache.
3      <name>Jakub 55621</name>
4      <description>
5          Weather App
6      </description>
7      <author email="wj55621@zut.edu.pl" href="https://www.wi.zut.edu.pl">
8          Jakub Wierciński
9      </author>
10     <content src="weather.html" />
11     <allow-intent href="http://*/*" />
12     <allow-intent href="https://*/*" />
13     <platform name="electron">
14         <icon src="res/electron/icon.png" />
15         <preference name="ElectronSettingsFilePath" value="res/electron/settings.json" />
16     </platform>
17 </widget>
18
```

Wstaw zrzut ekranu pliku PNG z ikoną o wymiarach 512x512px:



Wstaw zrzut ekranu uruchomionej aplikacji. Upewnij się, że wysokość i szerokość odpowiada wartościom z konfiguracji, pasek menu jest ukryty, ikonka jest zgodna ze skonfigurowaną, a narzędzia deweloperskie są zamknięte. **WAŻNE!!!** Użytkownicy MacOS – zrób zrzut ekranu w taki sposób, żeby ikonka uruchomionej aplikacji na pewno była widoczna – jeśli jest poza oknem – obejmij screenshotem cały ekran i zaznacz ikonkę.

LAB D - Pogoda REST API

Sprawdź pogodę

Pogoda

Punkty:	0	1
---------	---	---

BUDOWA APLIKACJI

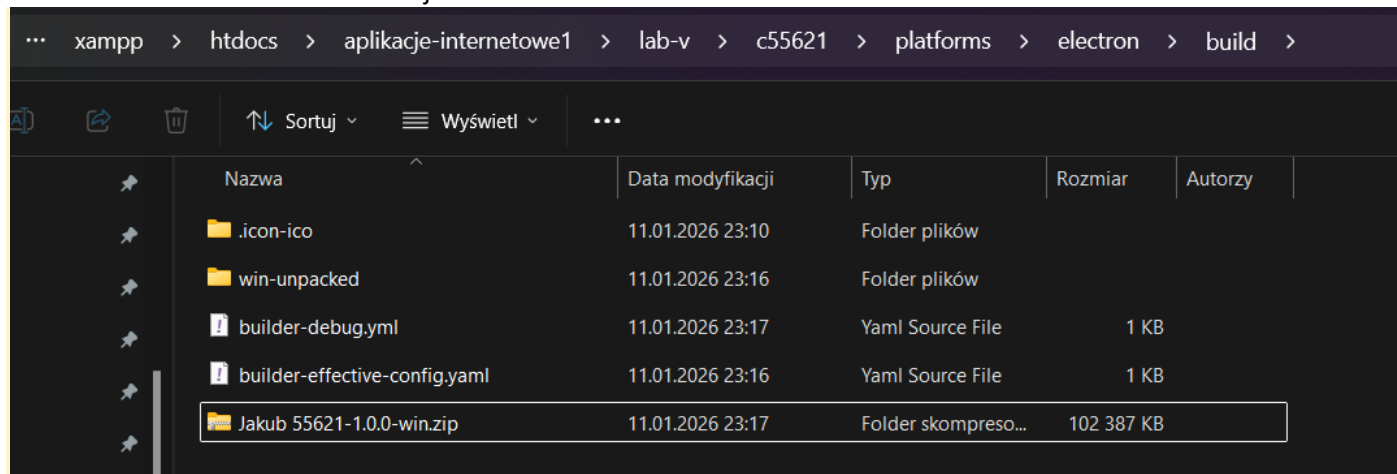
Wykonaj polecenie budowy aplikacji Electron:

```
> cordova build electron --release
```

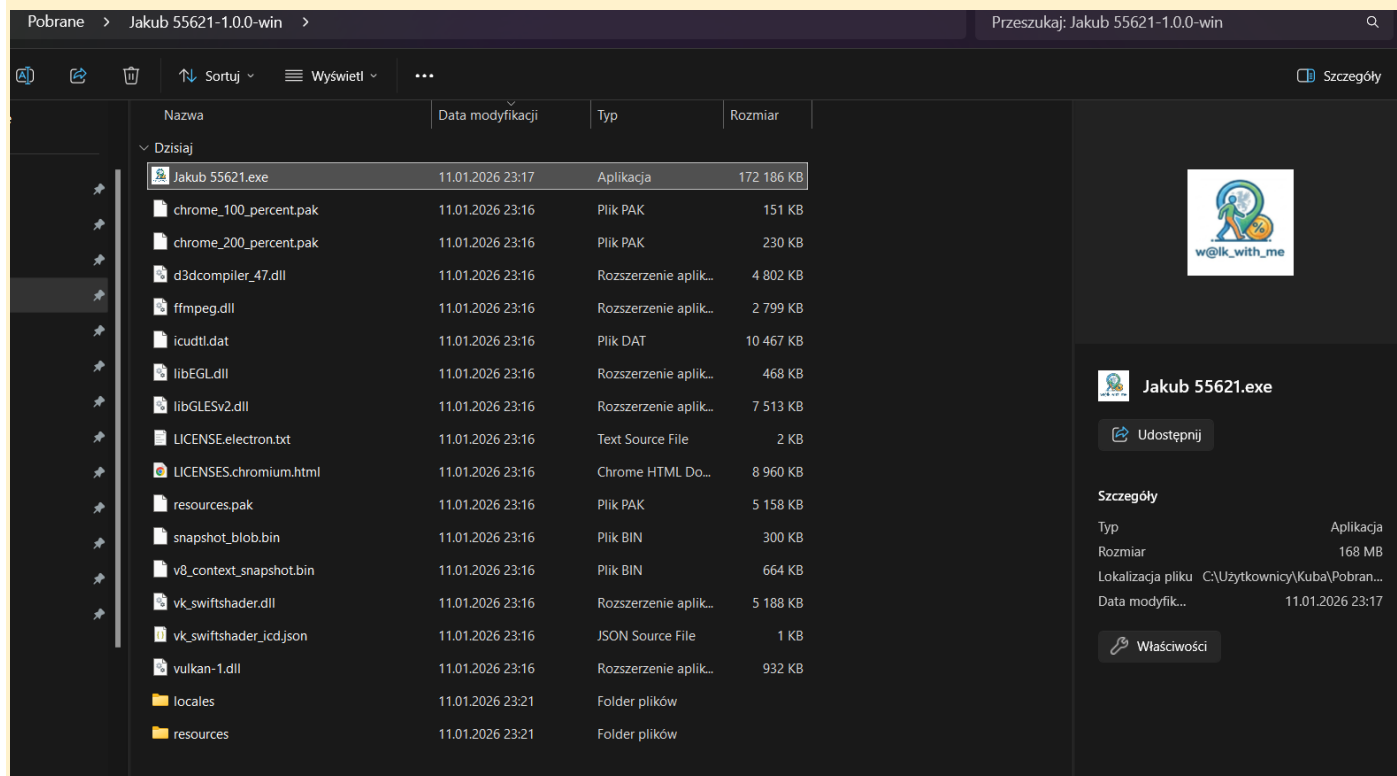
Odczekaj kilka minut. Po zakończonym procesie wynikowe archiwum ZIP znajdzie się w katalogu `\platforms\electron\build`.

Skopiuj archiwum do katalogu `Pobrane`. Rozpakuj archiwum do katalogu `Jakub 55621-1.0.0-win`, wejdź do środka rozpakowanego katalogu i uruchom plik EXE.

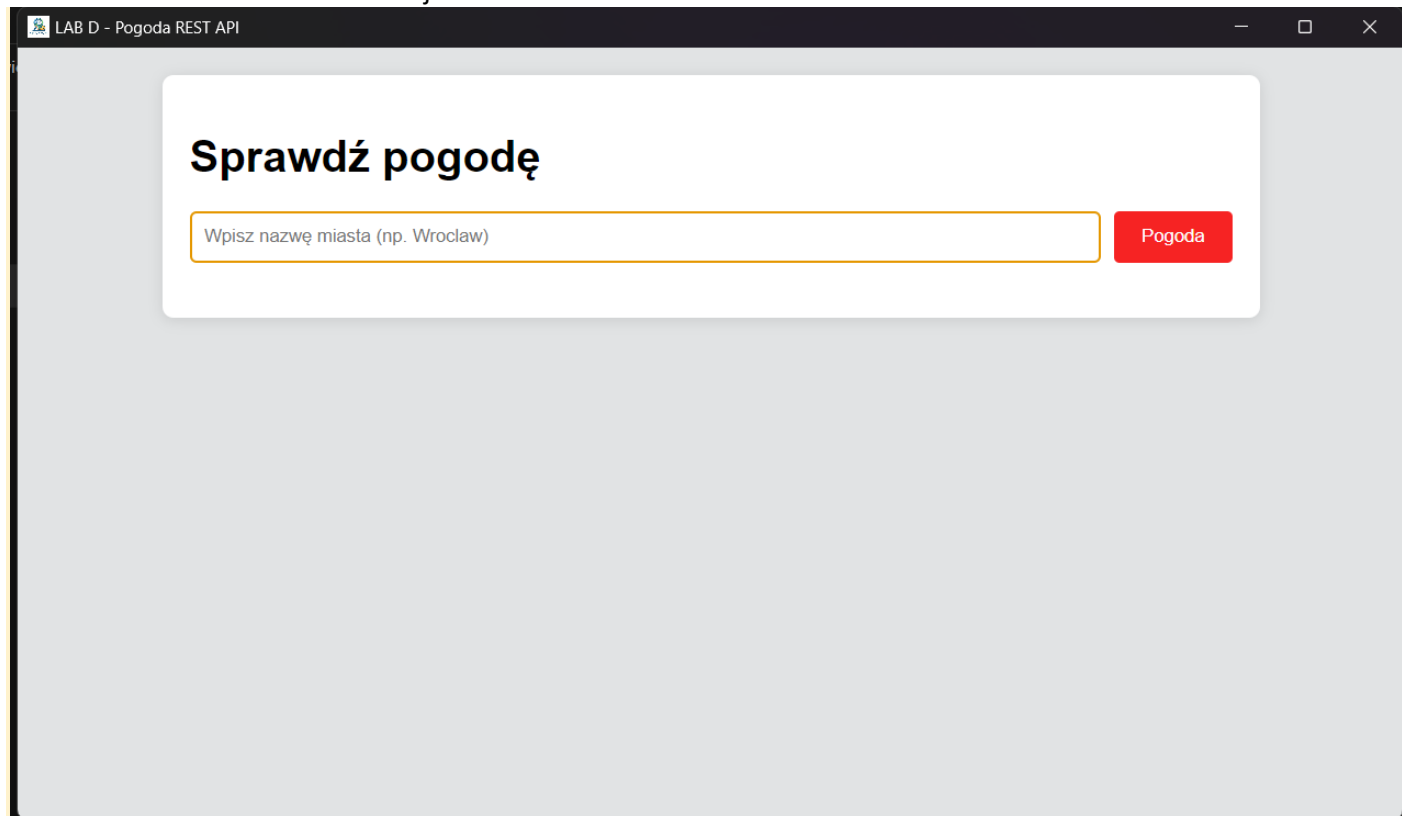
Wstaw zrzut ekranu zawartości katalogu `platforms\electron\build`. Zaznacz na zrzucie ekranu plik `Jakub 55621-1.0.0-win.zip`



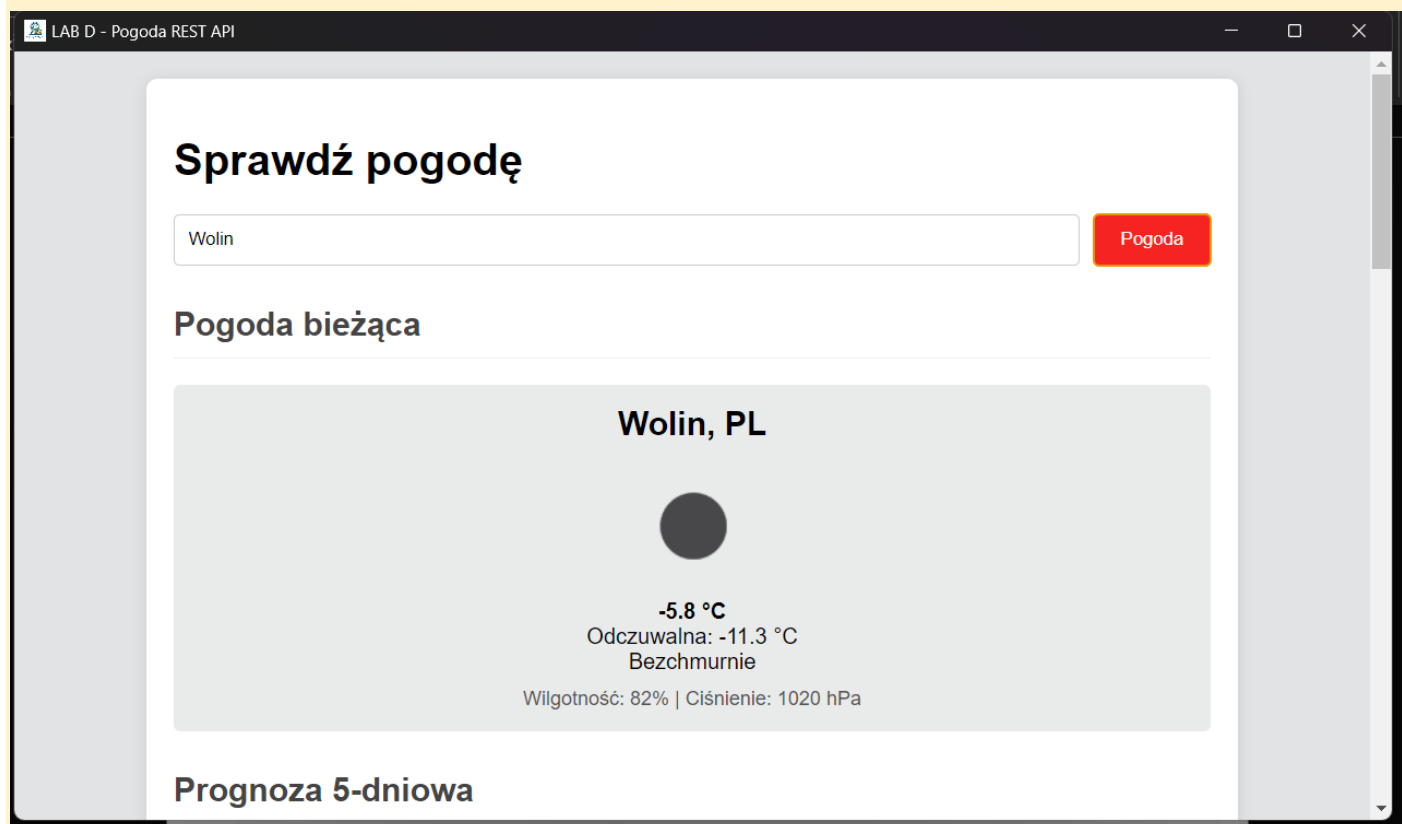
Wstaw zrzut ekranu zawartości rozpakowanego archiwum ZIP z aplikacją. Upewnij się, że na zrzucie ekranu jest ścieżka do katalogu. Zaznacz na zrzucie ekranu plik `Jakub 55621.exe`. Upewnij się, że przy pliku widoczna jest Twoja ikona.



Wstaw zrzut ekranu uruchomionej aplikacji przed sprawdzeniem pogody:



Wstaw zrzut ekranu uruchomionej aplikacji po sprawdzeniu prognozy pogody:



Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do **swojego** repozytorium GIT. **UWAGA! Nie commitować binarek!!!**

Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-v` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-v` w swoim repozytorium:

<https://github.com/kuba200333/aplikacje-internetowe1/tree/lab-v>

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.

Było to fajne zadanie i ciekawe, bo można było zrobić aplikację na komputer i telefon i było łatwe w skonfigurowaniu.

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.